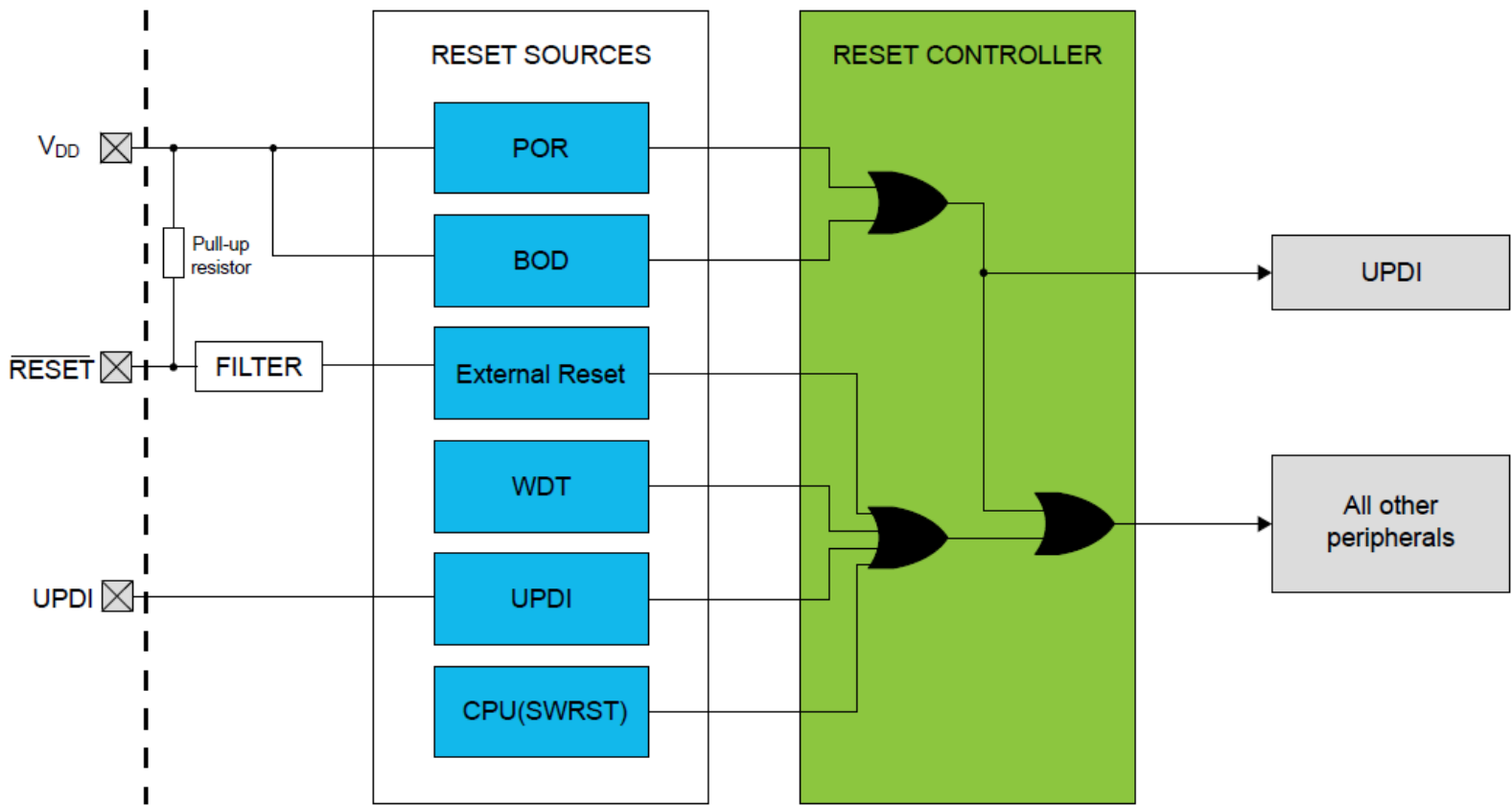
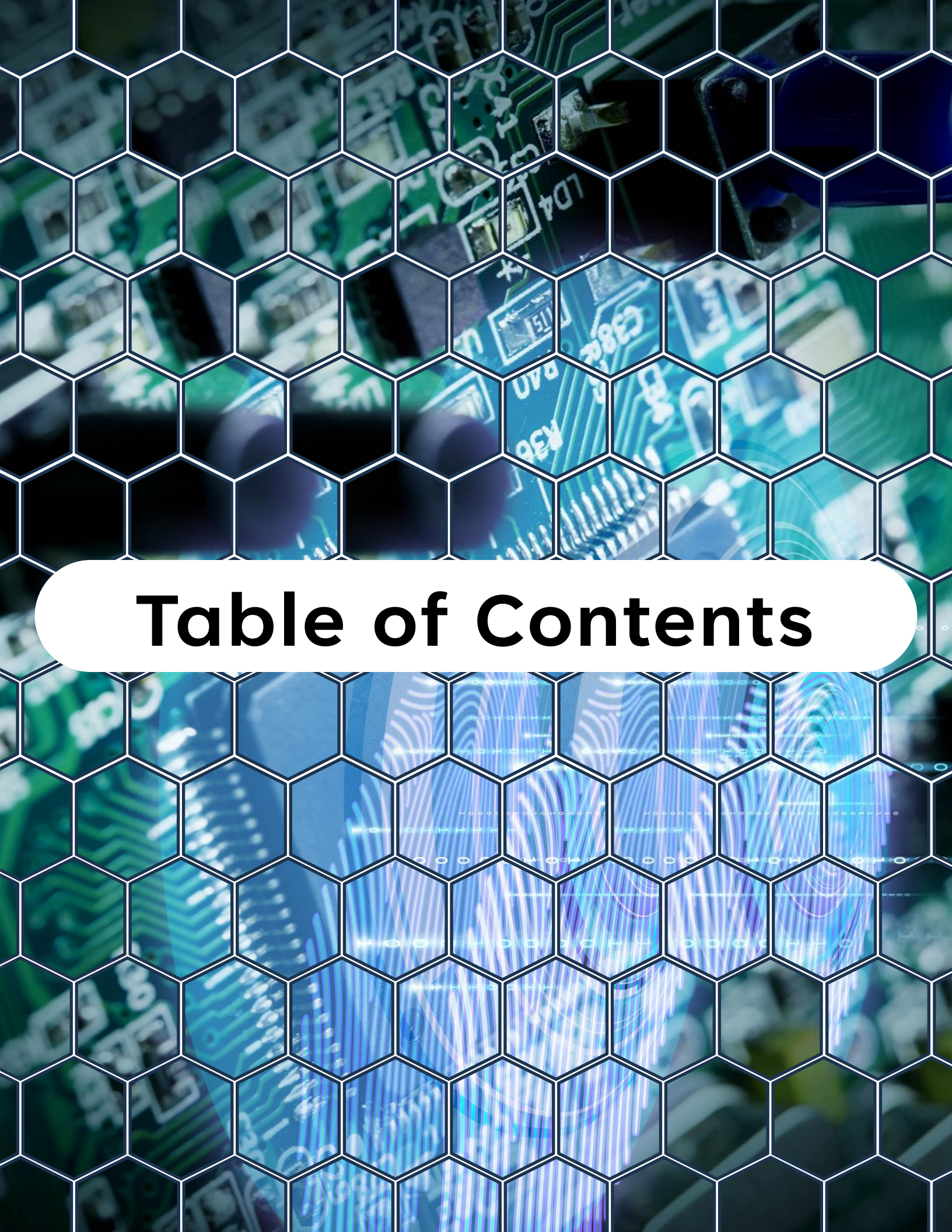


# “Bare-Metal Embedded Systems with AVR128DA48 Course”

## Day 5 - Reset System and Fuse Basics





# **Table of Contents**

# Table of Contents

1. Objective of Day 5
2. Reset System Overview
3. Reset Sources in AVR128DA48
4. Power-On Reset (POR)
5. Brown-Out Reset (BOD)
6. External Reset
7. Watchdog Reset
8. Software Reset
9. Reset Controller (RSTCTRL)
10. Reading the Reset Cause (Code Example)
11. Typical Uses of Reset Cause Information
12. What Are Fuses?
13. Common Fuse Settings (Conceptual)
14. Fuse Safety Rules

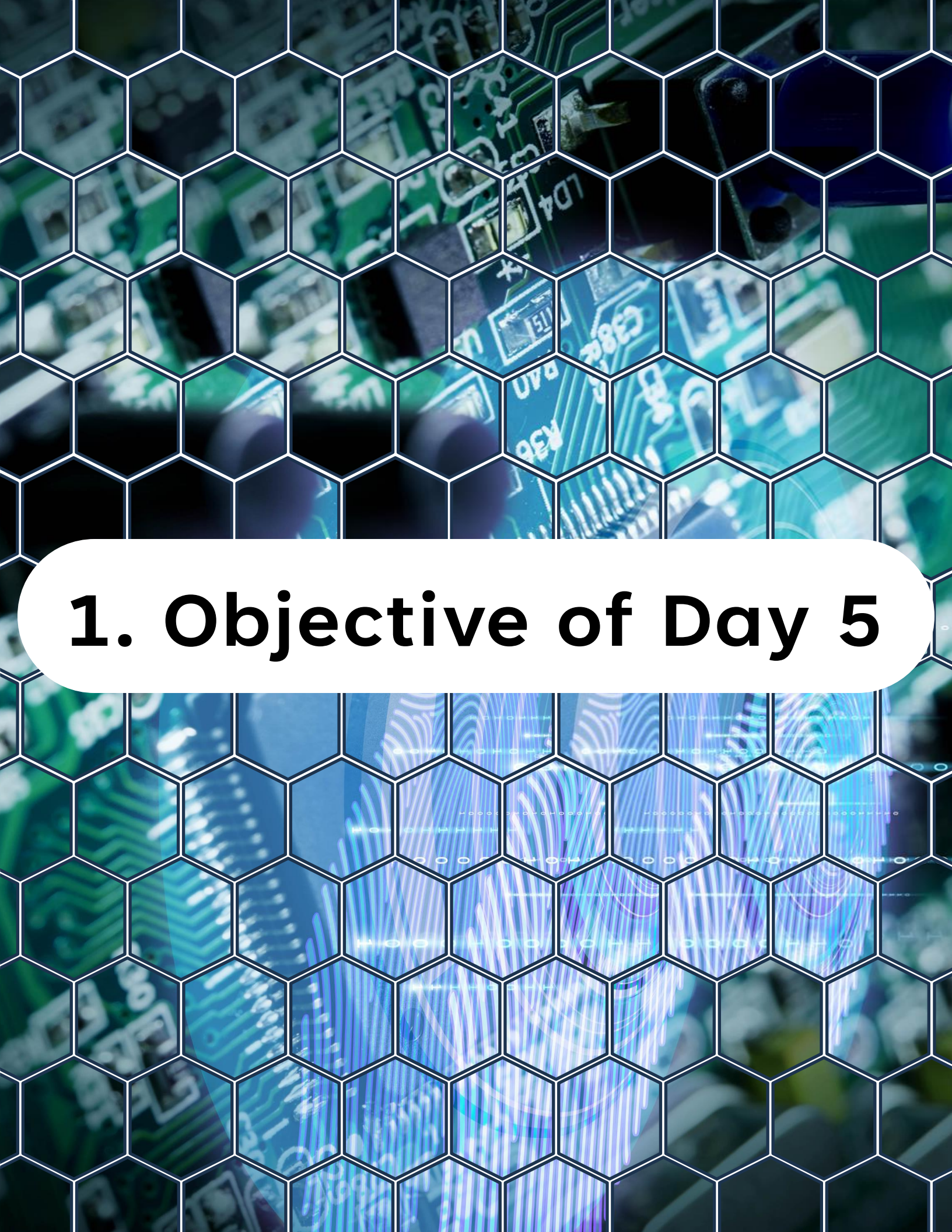
# Table of Contents

15. Practical Exercise - Reset Cause  
Experiment

16. Common Reset-Related Mistakes

17. What You Learned Today

18. What Comes Next



# **1. Objective of Day 5**

# 1. Objective of Day 5

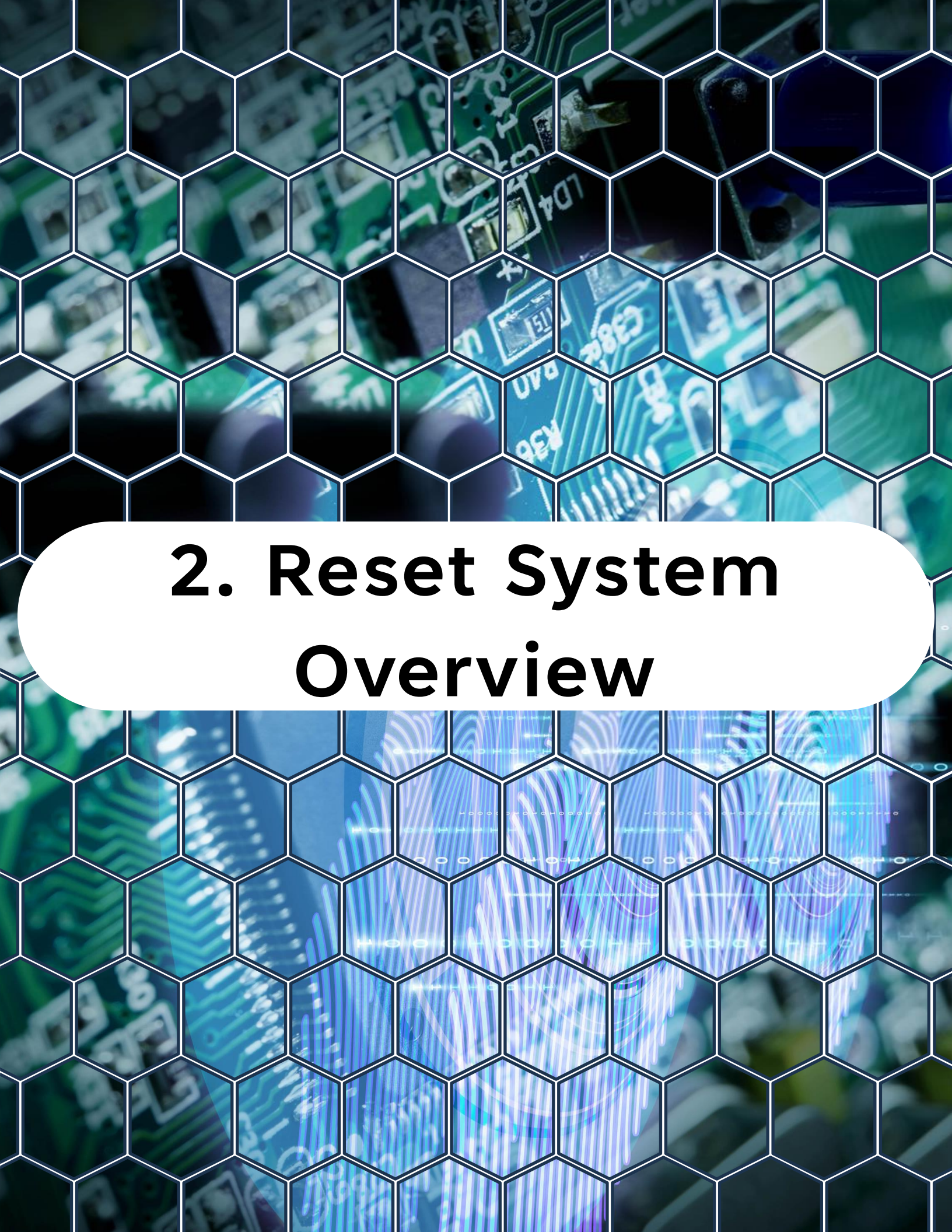
Reset behavior defines how your system starts, recovers from faults, and returns to a known safe state.

Today you will learn how resets actually work inside the MCU, how to detect why a reset occurred, and how fuses influence system behavior long before your C code runs.

By the end of this day, you will be able to:

- Understand all reset sources in the MCU
- Read and interpret reset cause flags
- Clear reset flags correctly
- Understand what fuses are and why they matter
- Avoid common reset and fuse-related mistakes

This is where firmware reliability begins.



## **2. Reset System Overview**

## 2. Reset System Overview

The **AVR128DA48** includes a centralized reset controller that monitors multiple reset sources and forces the CPU into a known startup state.

A reset:

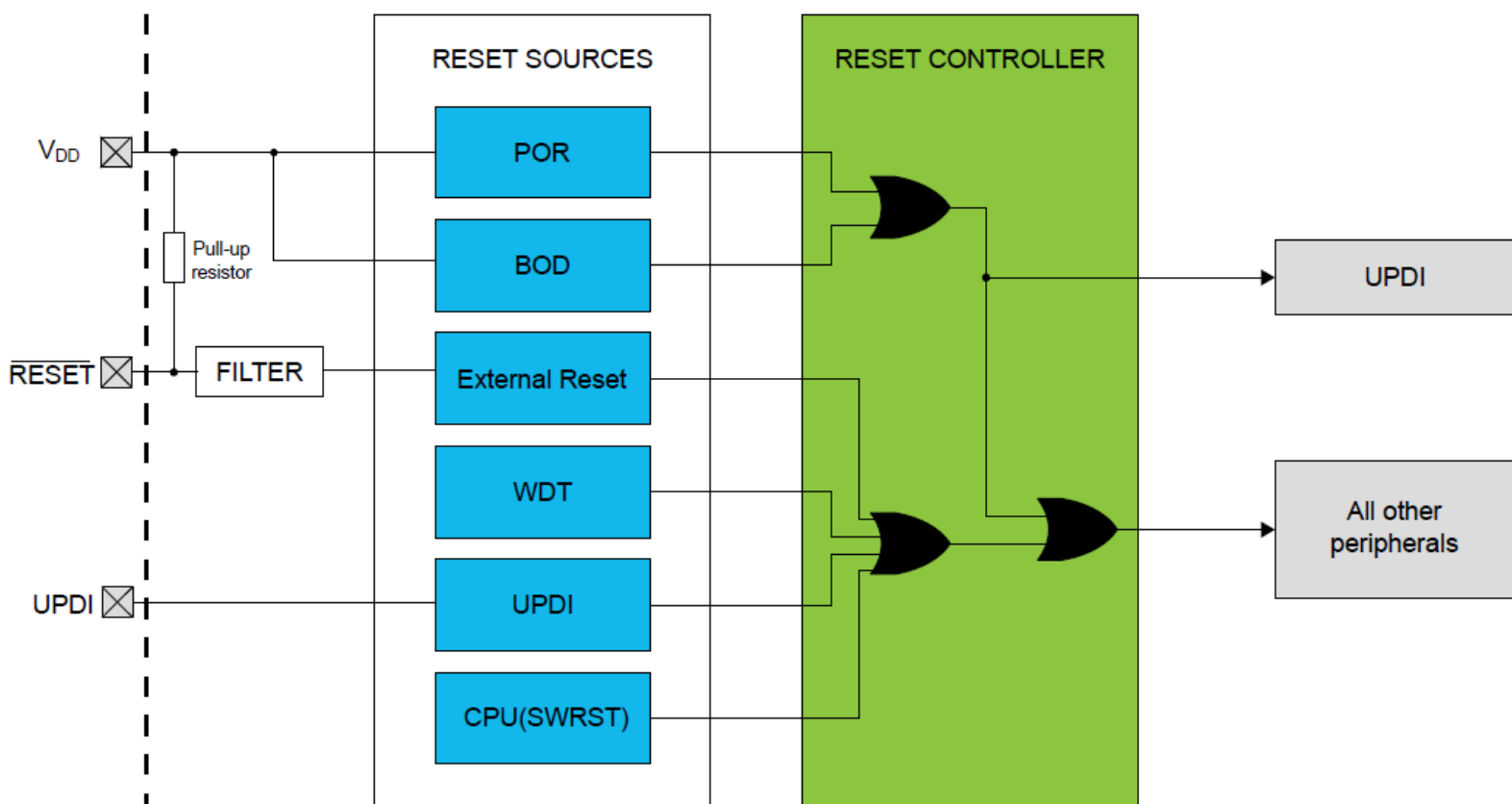
- Stops CPU execution
- Resets most peripheral registers
- Forces execution to restart from the reset vector
- Leaves a record of why the reset occurred

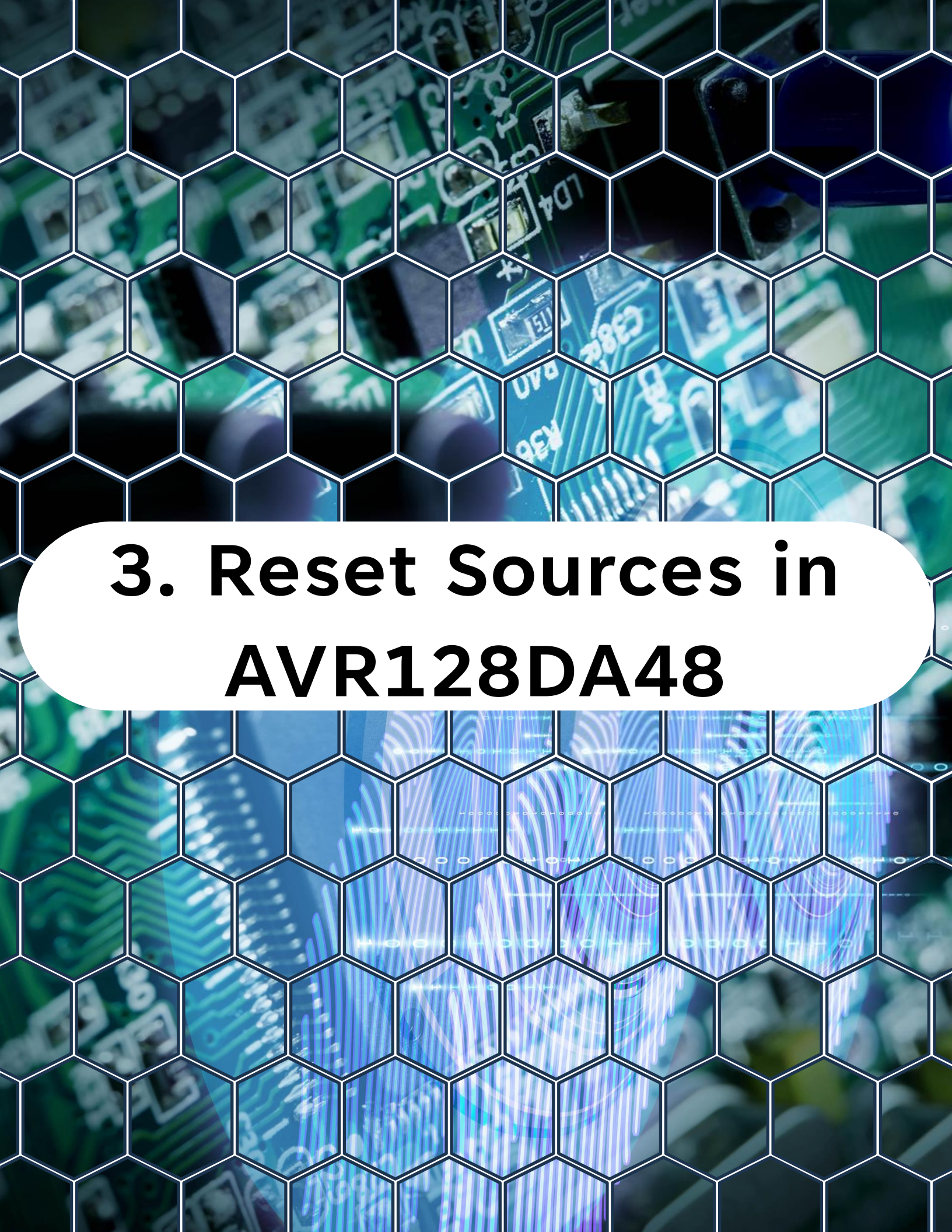
Resets are not failures. They are a normal and essential part of embedded systems.

## 2. Reset System Overview

### Reset System Overview

The Reset Controller (RSTCTRL) manages the Reset of the device. When receiving a Reset request, it sets the device to an initial state and allows the Reset source to be identified by the software. The Reset controller can also be used to issue a Software Reset (SWRST).





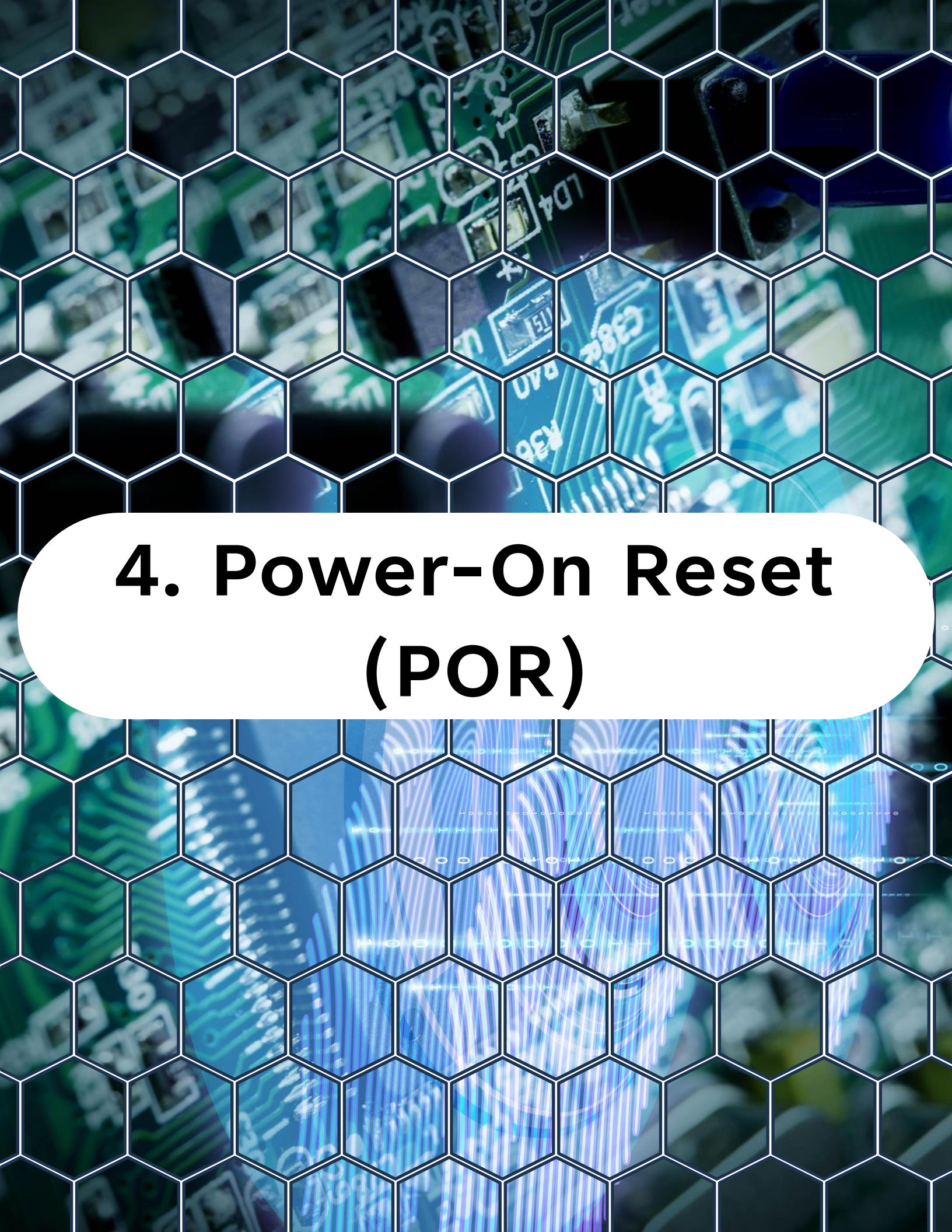
## **3. Reset Sources in AVR128DA48**

### **3. Reset Sources in AVR128DA48**

The MCU can reset from several sources, including:

- Power-On Reset (POR)
- Brown-Out Detector (BOD)
- External Reset
- Watchdog Timer (WDT)
- Software Reset
- Debug Reset

Each reset source sets a corresponding flag that firmware can inspect after startup.

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and resistors. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb-like texture. The text '4. Power-On Reset (POR)' is centered within a white, rounded rectangular box in the middle of the slide.

## **4. Power-On Reset (POR)**

## 4. Power-On Reset (POR)

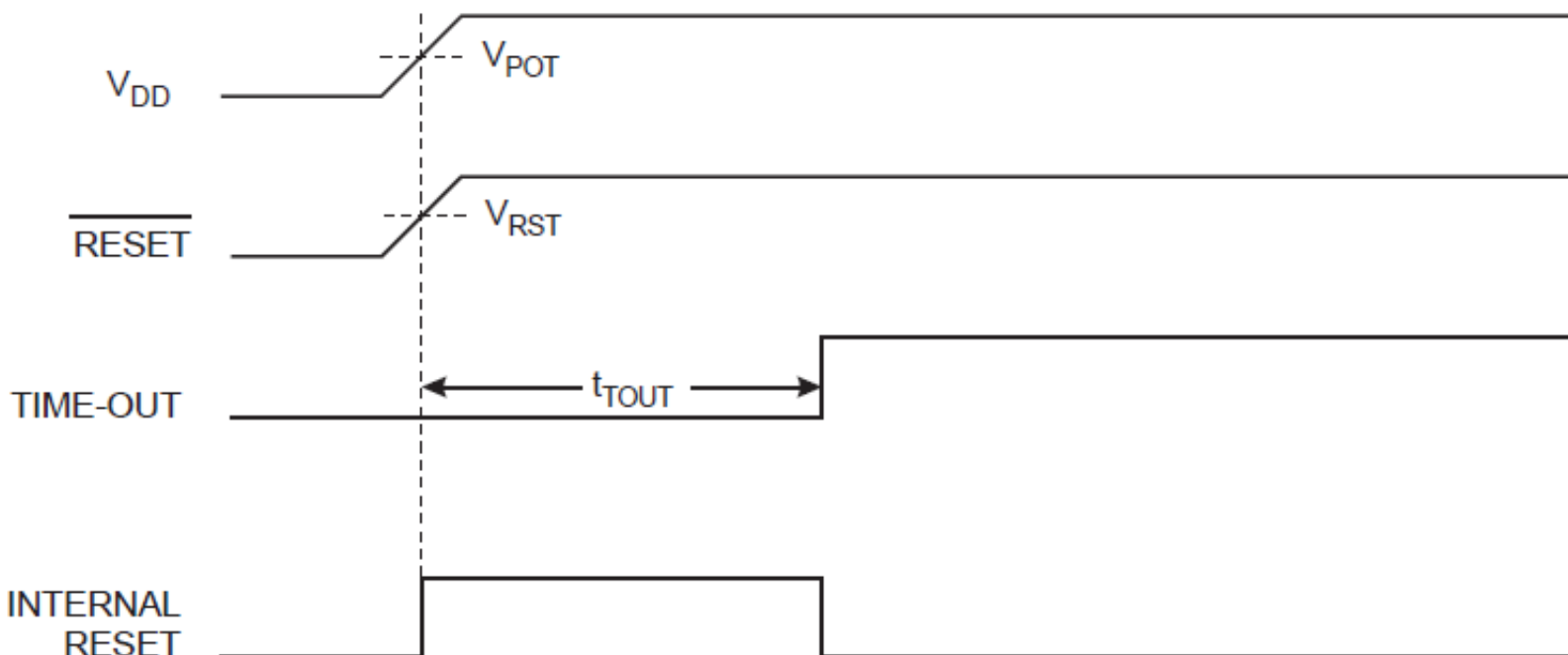
**Power-On Reset** occurs when:

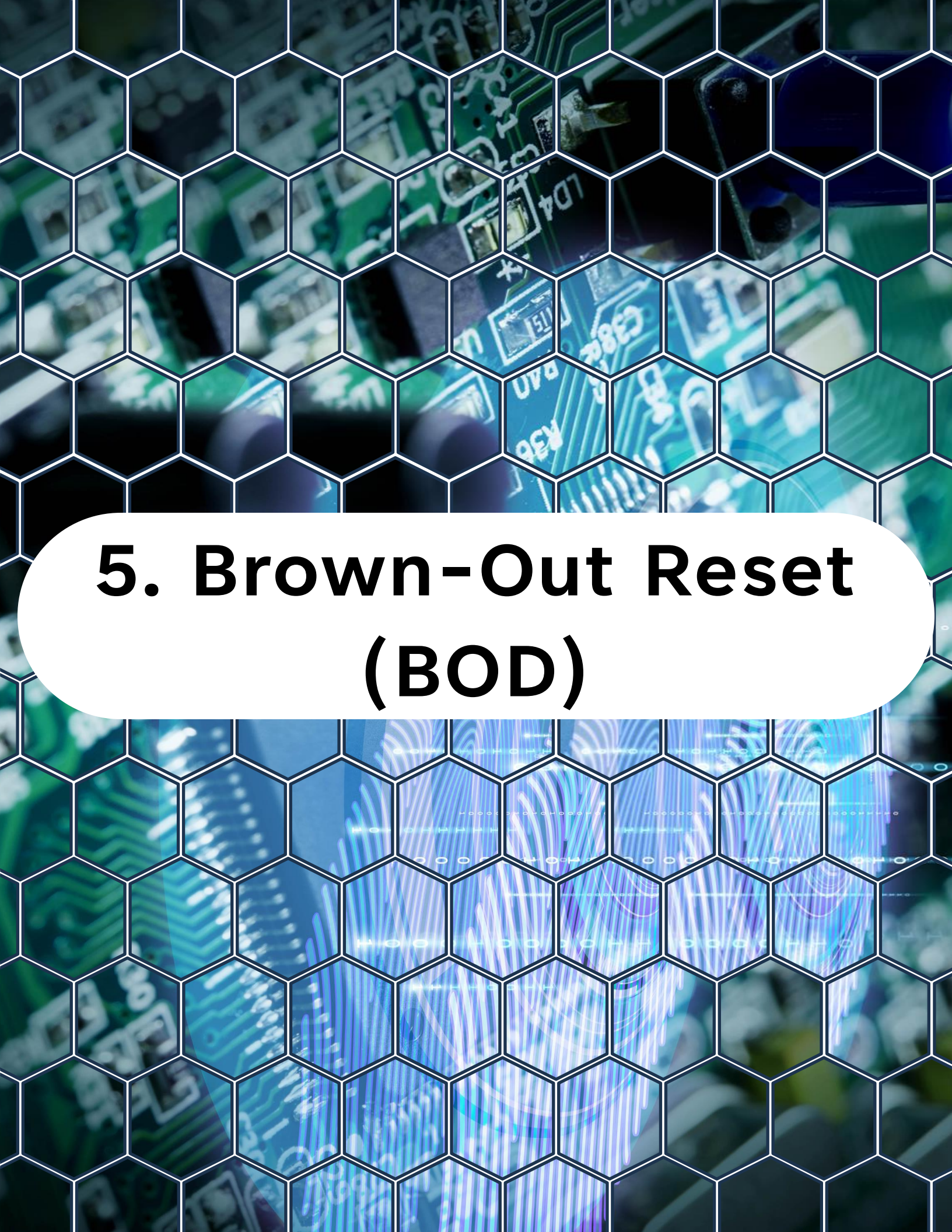
- $V_{DD}$  rises from 0 V to a valid operating voltage

POR ensures:

- CPU starts cleanly
- Registers are in a known state
- Flash execution begins safely

POR is automatic and cannot be disabled. It is your baseline reset condition.





## **5. Brown-Out Reset (BOD)**

## 5. Brown-Out Reset (BOD)

**Brown-Out** Reset protects the MCU from operating at unsafe voltages.

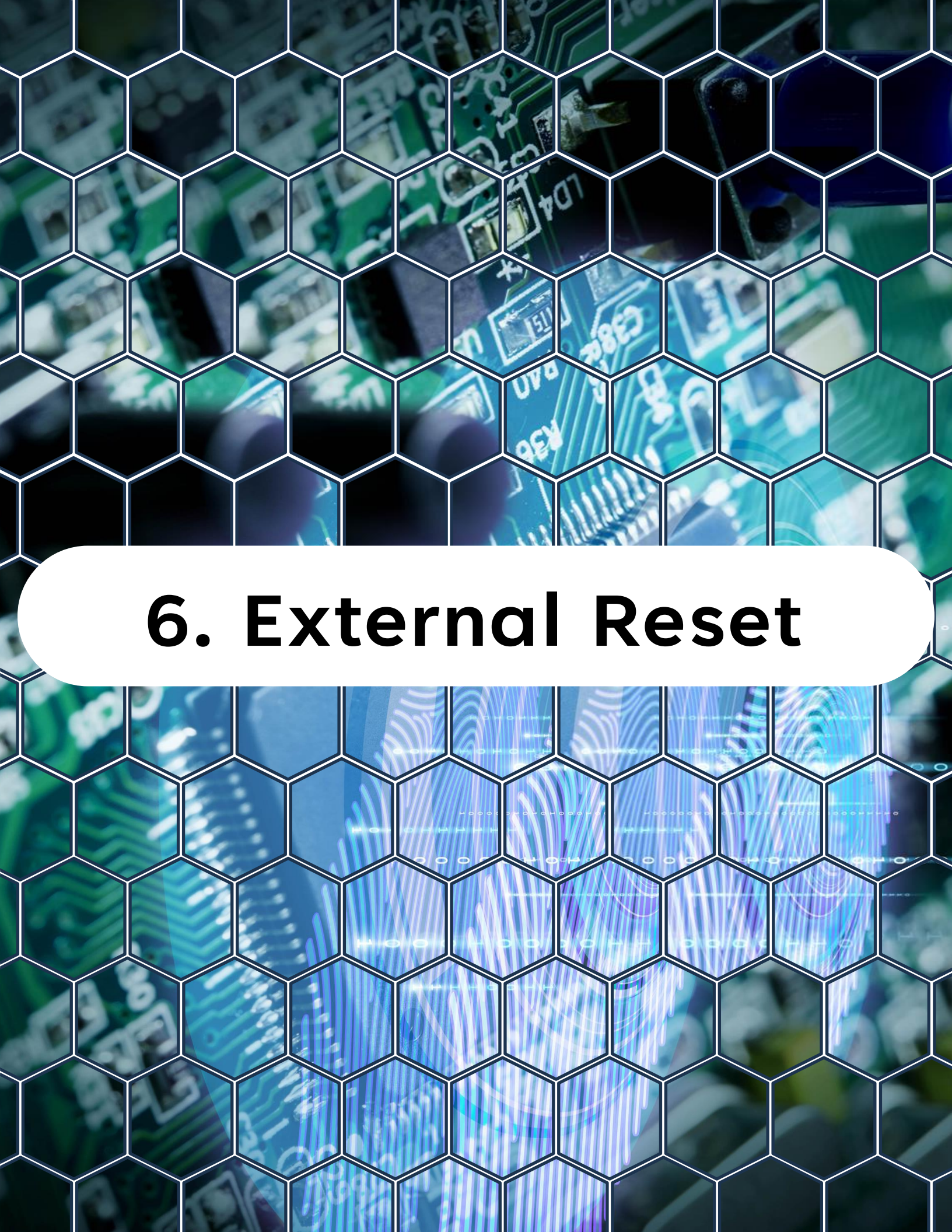
When enabled:

- The MCU monitors VDD
- If voltage drops below a threshold, a reset is triggered

Why this matters:

- Prevents corrupted memory writes
- Avoids unpredictable execution
- Improves long-term reliability

BOD behavior is controlled by fuses, not runtime code.



## **6. External Reset**

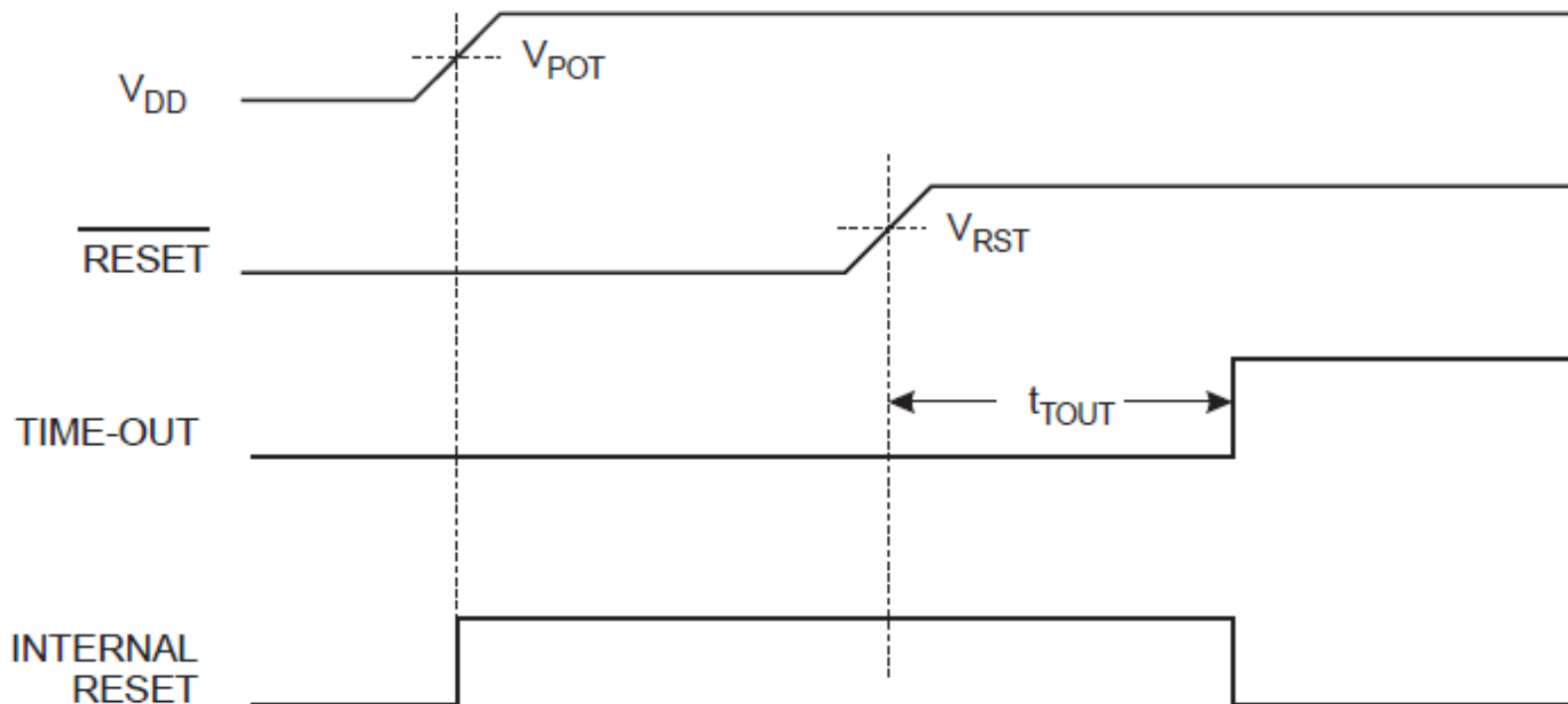
## 6. External Reset

External reset is triggered by:

- Pulling the RESET pin low (if enabled)
- External supervisor circuits

On many DA-series boards, RESET may be multiplexed or disabled in favor of UPDI. Always check the schematic and fuse configuration.

### MCU Start-Up, RESET Extended Externally





# **7. Watchdog Reset**

## 7. Watchdog Reset

The Watchdog Timer (WDT):

- Forces a reset if firmware stops responding
- Protects against lockups and runaway code

The Watchdog Timer (WDT) is a system function which monitors the correct operation of the program. If the WDT is not handled by software according to the programmed time-out period, a Watchdog Reset will be issued.

Watchdog resets are intentional safety mechanisms, not bugs.

You will explore the watchdog in depth later.

Today, you only need to recognize it as a reset source.



## **8. Software Reset**

## 8. Software Reset

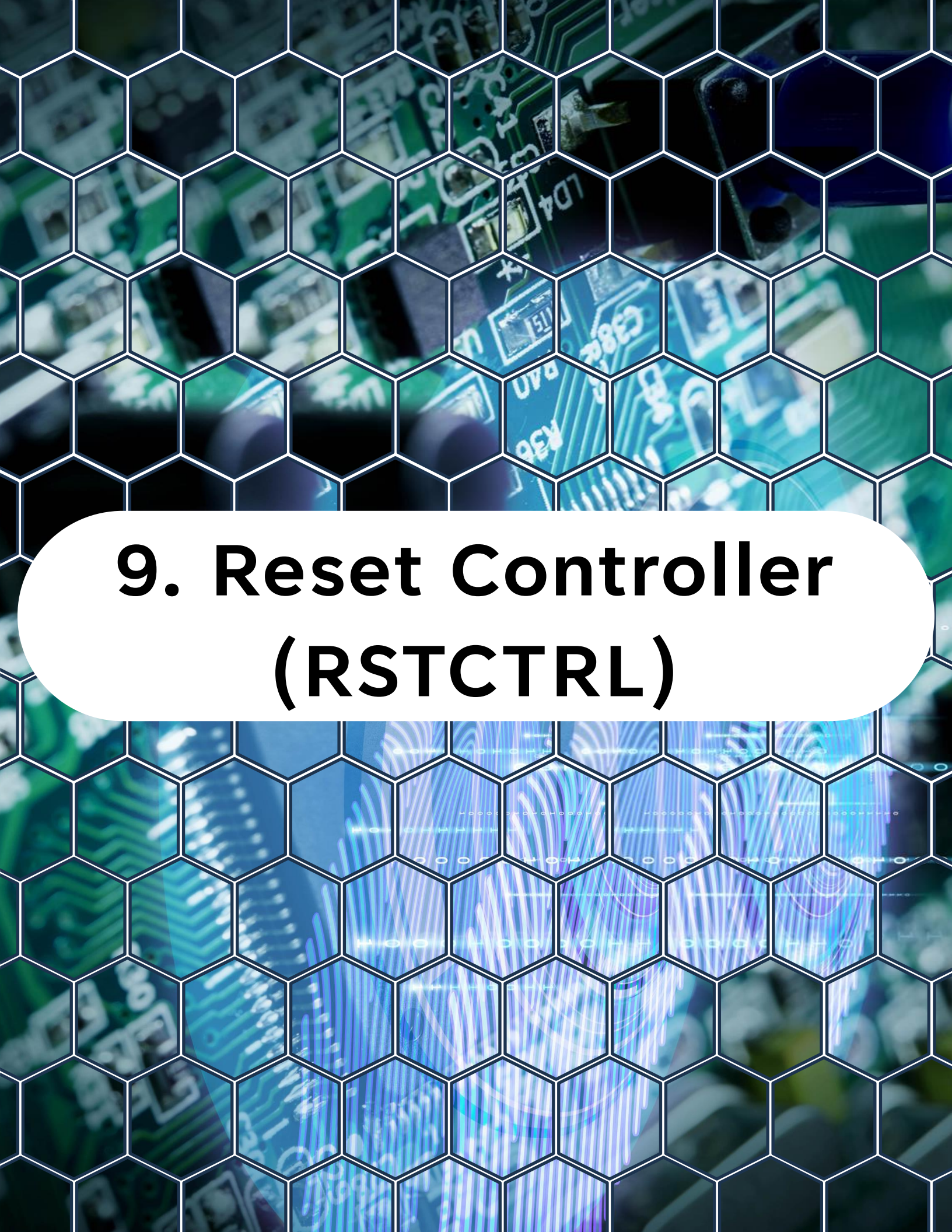
A software reset can be triggered intentionally by firmware.

Use cases:

- Apply configuration changes
- Recover from fatal errors
- Restart state machines cleanly

This reset behaves similarly to external reset but is fully controlled by code.

The software Reset makes it possible to issue a System Reset from the software. The Reset is generated by writing a '1' to the Software Reset (**SWRST**) bit in the Software Reset (**RSTCTRL.SWRR**) register.



## **9. Reset Controller (RSTCTRL)**

## 9. Reset Controller (RSTCTRL)

The reset system is managed by the **RSTCTRL** peripheral.

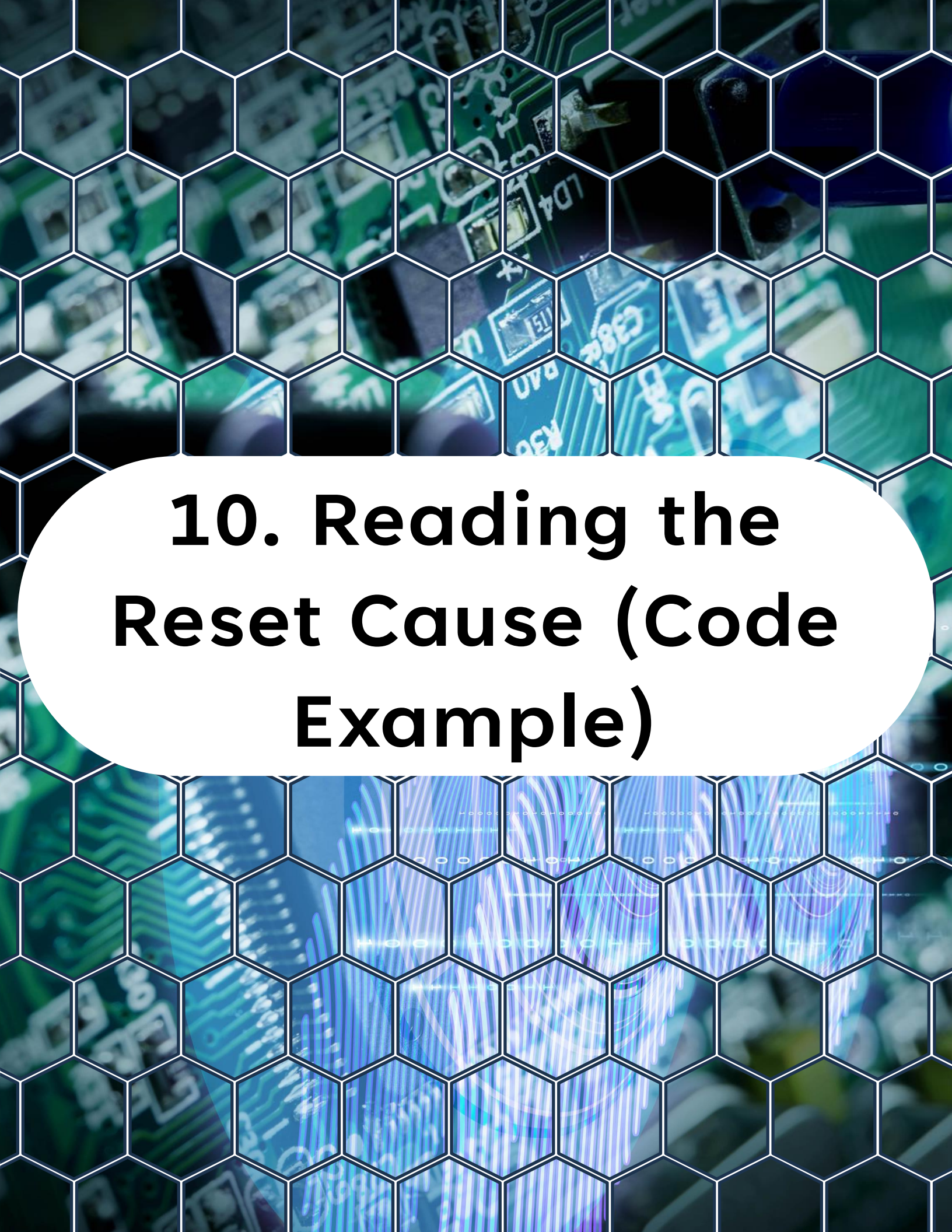
Key register:

- **RSTCTRL.RSTFR** (Reset Flag Register)

This register contains flags indicating which reset source caused the last reset.

Important rule:

- Reset flags are sticky
- You must clear them manually after reading



## **10. Reading the Reset Cause (Code Example)**

# 10. Reading the Reset Cause (Code Example)

A professional firmware reads the reset cause early during startup.

## Example: Reading and Clearing Reset Flags

```
1  /*
2   * Read reset cause early during startup (AVR128DA48, bare-metal)
3   *
4   * Why "early":
5   * - Reset flags can be overwritten/cleared by later init code.
6   * - You want the cause captured before anything else runs.
7   *
8   * Reset causes are stored in RSTCTRL.RSTFR.
9   * Best practice:
10  * - Read once into a variable
11  * - Clear the flags by writing 1s back to the same bits
12  */
13 #include <avr/io.h>
14 #include <stdint.h>
15
16 static volatile uint8_t g_reset_cause = 0;
17
18 /*
19  * Capture and clear the reset cause flags.
20  *
21  * Returns:
22  *   Bitmask of RSTCTRL.RSTFR flags at startup.
23  */
24 static inline uint8_t reset_cause_read_and_clear_early(void)
25 {
26     uint8_t flags = RSTCTRL.RSTFR; // Read reset flags (early snapshot)
27     RSTCTRL.RSTFR = flags;          // Clear by writing 1s to the set bits
```

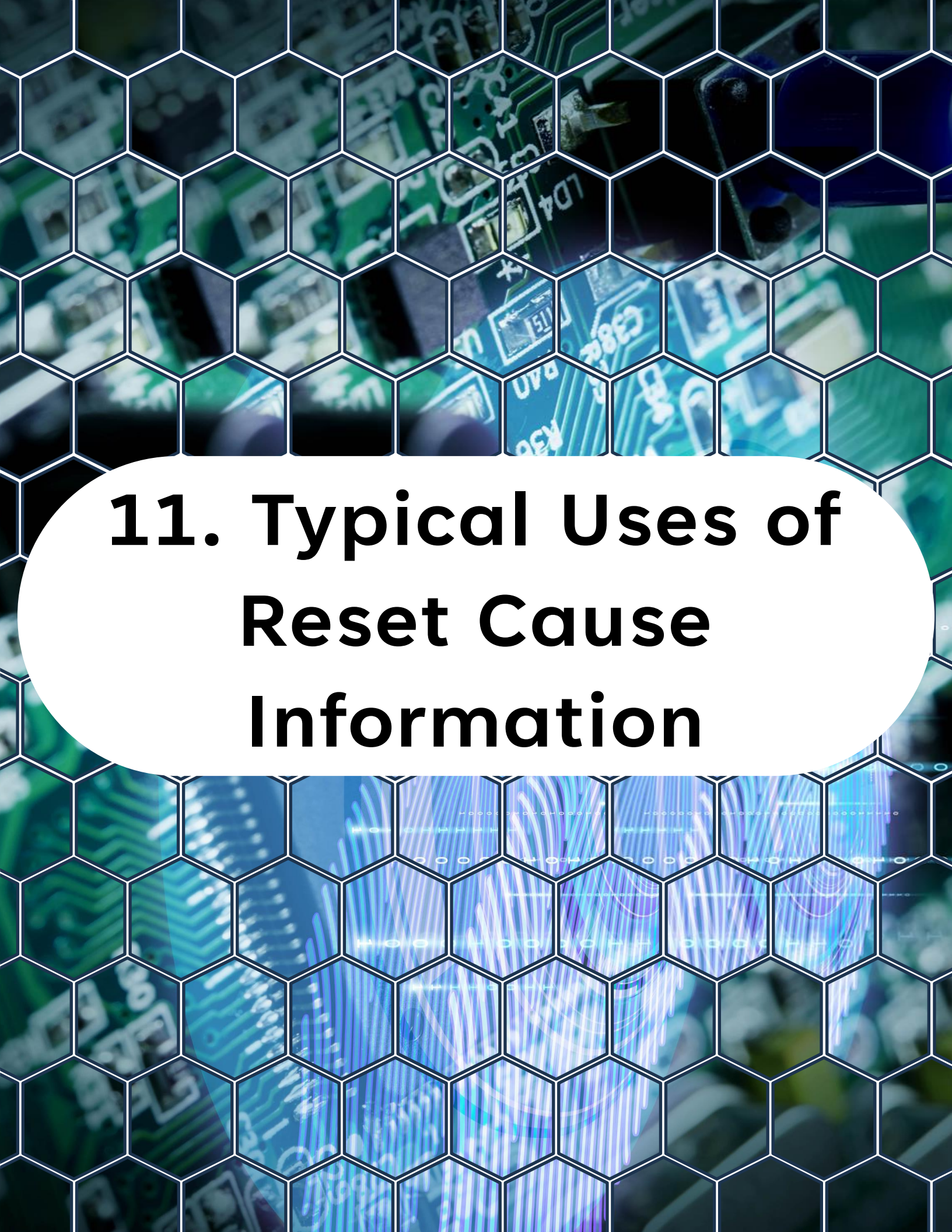
# 10. Reading the Reset Cause (Code Example)

```
24 static inline uint8_t reset_cause_read_and_clear_early(void)
25 {
26     uint8_t flags = RSTCTRL.RSTFR; // Read reset flags (early snapshot)
27     RSTCTRL.RSTFR = flags;          // Clear by writing 1s to the set bits
28     return flags;
29 }
30
31 int main(void)
32 {
33     /* Capture reset cause as the first thing in main(). */
34     g_reset_cause = reset_cause_read_and_clear_early();
35
36     /* Now continue with the rest of your bring-up... */
37     // clock_init_...();
38     // gpio_init_...();
39
40     while (1)
41     {
42         /* Your application */
43     }
44 }
```

## Explanation:

- RSTCTRL.RSTFR holds the reset reason
- Writing a 1 clears the corresponding flag
- Always clear flags after reading them

Later, you can use `reset_cause` for diagnostics or logging.



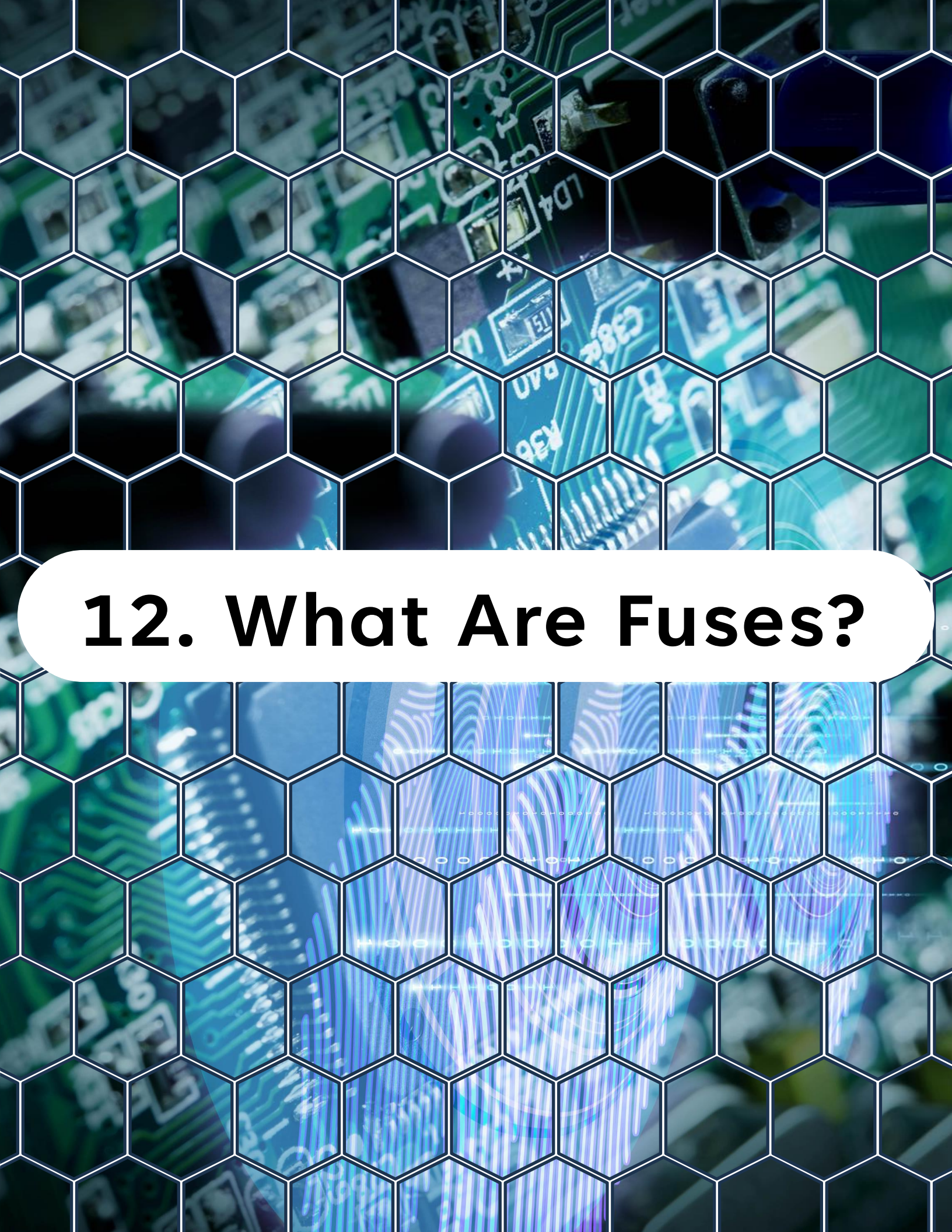
# **11. Typical Uses of Reset Cause Information**

# 11. Typical Uses of Reset Cause Information

Reset cause data is useful for:

- Debugging unexpected resets
- Detecting watchdog events
- Differentiating power loss vs software faults
- Implementing safe recovery behavior

In production systems, this information is often stored in EEPROM or sent over a debug interface.



## **12. What Are Fuses?**

## 12. What Are Fuses?

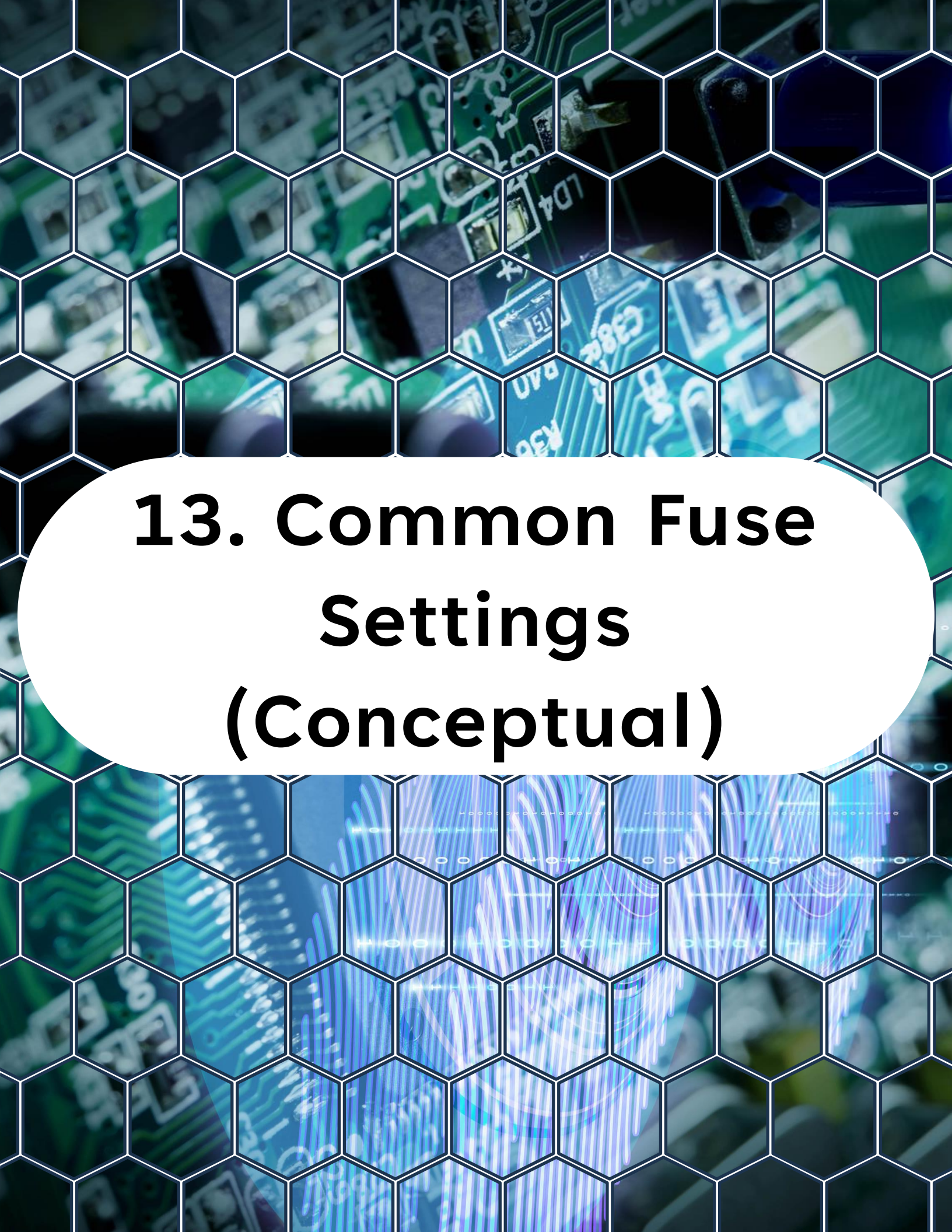
**Fuses** are non-volatile configuration bits that define MCU behavior before your program runs.

Important characteristics:

- Programmed outside normal Flash memory
- Evaluated at reset
- Control low-level behavior

Fuses are not registers. You cannot change them at runtime.

The fuses can be read by the CPU or the UPDI, but can only be programmed or cleared by the UPDI.



# **13. Common Fuse Settings (Conceptual)**

## **13. Common Fuse Settings (Conceptual)**

Typical fuse-controlled features include:

- Clock source selection
- Brown-Out Detector enable and threshold
- Watchdog behavior at startup
- Reset pin configuration
- UPDI enable

Changing a fuse can fundamentally alter how the MCU behaves.



# **14. Fuse Safety Rules**

# 14. Fuse Safety Rules

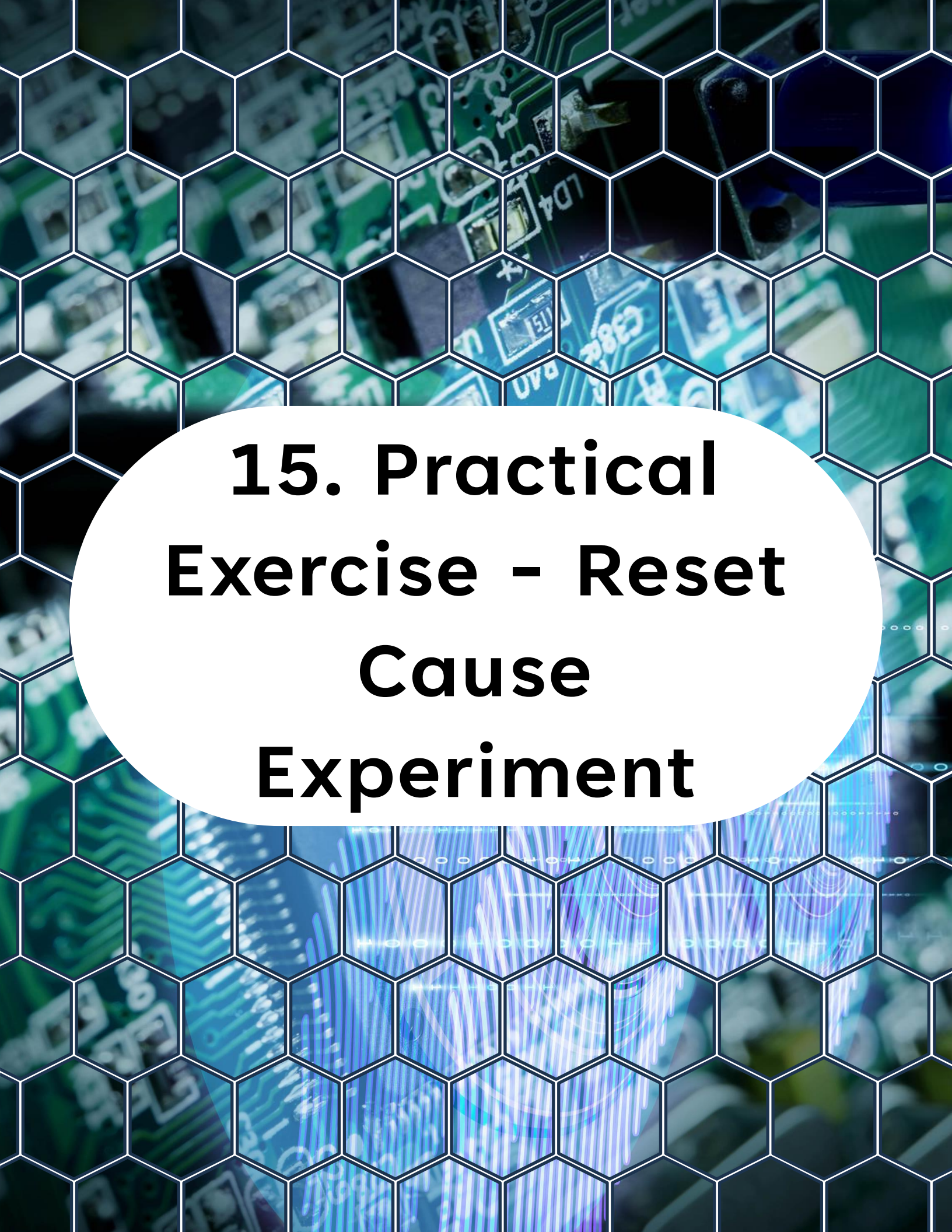
Follow these rules religiously:

- Never change fuses blindly
- Always read current fuse values first
- Change one fuse at a time
- Understand recovery options before programming

Incorrect fuse settings can:

- Lock you out of the device
- Disable programming interfaces
- Stop the MCU from running entirely

Respect fuses. They are powerful and unforgiving.



# **15. Practical Exercise - Reset Cause Experiment**

# **15. Practical Exercise - Reset Cause Experiment**

## **Exercise Goal**

Observe different reset sources and how they are reported.

## **Suggested Steps**

- Add reset cause code to your project
- Power-cycle the board and observe reset flags
- Trigger a debugger reset
- Later, compare with watchdog resets

Do not rush this. Understanding reset behavior early prevents painful debugging later.

# 15. Practical Exercise - Reset Cause Experiment

```
1  /*
2   * AVR128DA48 - Watchdog reset demonstration (bare-metal)
3   *
4   * Goal:
5   * - Intentionally trigger a watchdog reset
6   * - Read reset cause early and confirm WDT reset flag
7   *
8   * Notes:
9   * - On AVR DA-series, WDT is controlled by WDT.* registers.
10  * - WDT configuration is protected (CCP).
11  * - After reset, the reset cause is available in RSTCTRL.RSTFR.
12  */
13 #include <avr/io.h>
14 #include <stdint.h>
15 #include <avr/interrupt.h>
16
17 /* Store reset cause for debugging (watch in debugger) */
18 static volatile uint8_t g_reset_cause = 0;
19
20 /* Protected register write helper */
21 static inline void ccp_write_io(volatile uint8_t *addr, uint8_t value)
22 {
23     CCP = CCP_IOREG_gc;
24     *addr = value;
25 }
26
27 /*
28  * Capture and clear reset flags as early as possible.
29  *
30  * Returns:
31  *   Bitmask of reset flags present at startup.
32  */
33 static inline uint8_t reset_cause_read_and_clear_early(void)
34 {
35     uint8_t flags = RSTCTRL.RSTFR; // Read
36     RSTCTRL.RSTFR = flags;          // Clear by writing back the set bits
37     return flags;
38 }
39
40 /*
41  * Enable watchdog with a short timeout.
```

# 15. Practical Exercise - Reset Cause Experiment

```
40 /*
41  * Enable watchdog with a short timeout.
42  * Intentionally do NOT feed it afterward to force a reset.
43  *
44  * Timeout selection:
45  * - PER_1KCLK is a common short period choice.
46  * - Exact time depends on WDT clock source (datasheet), but it will reset quickly.
47  */
48 static void wdt_enable_short_timeout(void)
49 {
50     // Disable global interrupts.
51     cli();
52     // Reset Watchdog Timer.
53     __asm__ __volatile__ ("wdr");
54     // Enable WDT, set a short period
55     ccp_write_io(&WDT.CTRLA, WDT_WINDOW_OFF_gc | WDT_PERIOD_1KCLK_gc);
56     // Enable global interrupts.
57     sei();
58 }
59
60 int main(void)
61 {
62     /* Step 1: read reset cause right away */
63     g_reset_cause = reset_cause_read_and_clear_early();
64
65     /* If the previous reset was caused by WDT, this bit should be set */
66     /* Typical name in headers is RSTCTRL_WDRF_bm (watchdog reset flag). */
67
68     /* Step 2: enable watchdog */
69     wdt_enable_short_timeout();
70
71     /* Step 3: hang forever WITHOUT clearing the WDT -> watchdog reset occurs */
72     while (1)
73     {
74         /* Intentionally do nothing.
75          * No wdt_reset(), no feeding, no delays.
76          * The MCU will reset when the WDT expires.
77          */
78     }
79 }
```

The background of the slide features a close-up, slightly blurred image of a green printed circuit board (PCB). The board is populated with various electronic components, including integrated circuits and surface-mount components. A white hexagonal grid pattern is overlaid on the entire image, creating a honeycomb effect. In the center, a white rounded rectangle contains the title text.

# **16. Common Reset-Related Mistakes**

## 16. Common Reset-Related Mistakes

Avoid these errors:

- Ignoring reset cause flags
- Assuming all resets are power-related
- Forgetting to clear reset flags
- Changing fuses without documentation
- Debugging symptoms instead of root causes

Many "random bugs" are actually reset problems.



## **17. What You Learned Today**

# 17. What You Learned Today

By the end of Day 5, you should understand:

- All major reset sources
- How the reset controller works
- How to read and clear reset flags
- What fuses are and why they matter
- How resets fit into reliable system design

You now understand how the MCU starts and restarts.



# **18. What Comes Next**

# 18. What Comes Next

## Week 2 - Timers and Timekeeping

Starting with **Day 6 - Introduction to Timers on AVR DA**, you will move from basic MCU control into precise time-based behavior.

From this point on:

Code will become event-driven

Polling will be replaced with timing

Firmware will start to feel like a real system

When you are ready, we move on to Day 6.

*"Thanks for watching!*

*If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content.*

*Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"*

<https://www.linkedin.com/in/yamil-garcia>

<https://www.youtube.com/@LearningByTutorials>

<https://github.com/god233012yamil>